



CSCI 112

Programming with C

LECTURE: 5

J. L. Pach

OUTLINE:

Control flow:

- ✂ if-else statement
- ✂ switch statement
- ✂ goto statement

CONTROL

Flow

IF-ELSE STATEMENT 1/4

```
if (expression1)
    single_statement1
else
    single_statement2
```

- ⚡ `expression1` is non-zero $\Rightarrow$ statement1
- ⚡ `expression1` could be series of expressions
- ⚡ Block can be substitute for `simple_statement`
- ⚡ `else` is optional

IF-ELSE STATEMENT 2/4

```
int main()
{
    int x = 5;
    if( x > 1 )
        printf("%s\n", "a variable x is greater than 1");
}
```



a variable x is greater than 1

- ✂ Why is an if-else statement considered a statement when it doesn't change anything on its own?
- ✂ It's not true that it doesn't change the PC, but to explain that we need to understand ternary operator and labels.

IF-ELSE STATEMENT 3/4

✂ With optional `else`

```
int main()
{
    int x = 5;
    if( x > 1 )
        printf("%s\n", "a variable x is greater than 1");
    else
        printf("%s\n", "a variable x is NOT greater than 1");
}
```



a variable x is greater than 1

IF-ELSE STATEMENT 4/4

```
int main()
{
    int x = 0;
    if( x > 1 )
    {
        printf("%s", "a variable x");
        printf("%s\n", " is greater than 1");
    }
    else
    {
        printf("%s", "a variable x");
        printf("%s\n", " is not greater than 1");
    }
}
```



a variable x is not greater than 1

NESTED IF-ELSE STATEMENT 1/3

- ✖ Since an `if` statement following its condition in parentheses (the expression that, if it evaluates to a non-zero value, executes the subsequent code block) is considered a statement itself;
- ✖ it's important to note that another if statement can be placed after either the code block of the first `if` or its `else` block (if present).

NESTED IF-ELSE STATEMENT 2/3

```
int main()
{
    int x = 1;

    if(x == 0)
    ;
    else if(x == 1)
    ;
    else if(x > 2)
    ;
    else
    ;
}
```



```
int main()
{
    int x = 1;
    if(x == 0)
    {
    }
    else
    if(x == 1)
    {
    }
    else
    if(x > 2)
    {
    }
    else
    {
    }
}
```



NESTED IF-ELSE STATEMENT 3/3

```
int main()
{
    int x = getche() - 48;
    printf("%s", "a variable x is ");

    if(x == 0)
    {
        printf("%s\n", "0");
    }
    else
    {
        if(x == 1)
        {
            printf("%s\n", "1");
        }
        else
        {
            if(x > 1 && x < 5)
            {
                printf("%s\n", "between 2 and 4");
            }
            else
            {
                printf("%s\n", "not between 0 and 4");
            }
        }
    }
}
```



```
int main()
{
    int x = getche() - 48;
    printf("%s", "a variable x is ");

    if(x > 0)
    if(x < 5)
        printf("%s\n", "between 1 and 4");

}
```



SWITCH

statement

SWITCH STATEMENT 1/1

```
switch (expression)
{
    case <constant-expression1>:
        /* code */
        break;

    case <constant-expression2>:
        /* code */
        break;

    default:
        /* code */
        break;
}
```



```
int main()
{
    int x = getche() - 48 ; // typed 4
    printf("%s", "a variable x is ");
    switch (x) /* constant-expression */
    {
        case 1:
            printf("%s\n", "1");
            break;
        case 2:
            printf("%s\n", "2");
            break;
        default:
            printf("%s\n", "not between 1 and 3"); // <- true
            break;
    }
} // result: "a variable x is not between 1 and 3"
```

SWITCH STATEMENT 1/2

In C programming, a **constant expression** is an expression that evaluates to a fixed value at compile time, meaning the value is determined during the compilation process rather than during program execution.

- ✂ The `default` block in the switch statement is optional!
- ✂ The `default` block can be used for error handling.
- ✂ It's important to note that omitting the `default` block will cause the program to skip the entire switch statement if the variable takes on a value not defined in the switch statement.
- ✂ To terminate a switch statement, we use the `break` keyword

SWITCH STATEMENT 1/3

```
int main()
{
    int x = getche() - 48; //typed 2
    printf("%s", "a variable x is ");
    switch (x) /* constant-expression */
    {
        case 1:
            printf("%s\n", "1");
            break;
        case 2:
            printf("%s\n", "2");
            break;
    }
} // result: "a variable x is 2"
```



SWITCH STATEMENT 1/4

```
int main()
{
    int x = getche() - 48; // typed 2
    printf("%s", "a variable x is ");
    switch (x) /* constant-expression */
    {
        case 1:
        case 2:
        case 3:
            printf("%s\n", "between 1 and 3");
            break;
        default:
            printf("%s\n", "not between 1 and 3");
            break;
    }
} // result: "a variable x is 1 and 3"
```



```
int main()
{
    int x = getche() - 48; // typed 2
    printf("%s", "a variable x is ");
    switch (x) /* constant-expression */
    {
        case 1:
            printf("%s", "between");
        case 2:
            printf("%s", " 1 and");
        case 3:
            printf("%s\n", " 3");
            break;
        default:
            printf("%s\n", "not between 1 and 3");
            break;
    }
} // result: "a variable x is 1 and 3"
```



DIFFERENCE BETWEEN SWITCH & IF-ELSE

✂ **if** statements:

✂ **Dynamic Conditions:** if statements allow for dynamic conditions that can change during program execution. These conditions can involve complex expressions, function calls, and even user input.

✂ **switch** statements:

✂ **Static Values:** switch statements are primarily designed for evaluating expressions against a set of predefined, static values. These values are typically determined before program execution.

GOTO

statement

GOTO STATEMENT 1/2

```
int main()
{
    bool is_true = true;
    if ( is_true )
        goto label1;
    else
        goto label2;

    label1:
        printf("It was true\n");
        goto end;
    label2:
        printf("It was false\n");
        goto end;
    end:
} // result: It was true
```



GOTO STATEMENT 2/2

- ✂ goto allows us to jump (within the same function) unconditionally to a location identified by a `label`
- ✂ a simple example of Spaghetti Code - Dijkstra, E. W. (1968)

THANK

You